

UNAM

Universidad Nacional Autónoma de México

Facultad de Ciencias

Modelado y Programación

Práctica 04

Equipo Polimórfo

Juárez Larrondo Omar Alejandro
(322245244)

Soto Mendoza Ismael de Jesús
(322210422)

Vega Navas Saúl (322088267)

Identificación de la Práctica

Este documento corresponde a la **Práctica 04** de la materia *Modelado y Programación* de la Facultad de Ciencias, UNAM. El objetivo principal es implementar adecuadamente los patrones de diseño **Composite** y **Adapter** en la resolución del problema propuesto para el sistema de catálogo de la cadena de renta de películas RockBuster.

1.1. Descripción del Problema

Se implementó un sistema integral de gestión de catálogo para la cadena de renta de películas RockBuster, donde se organizan productos disponibles para consulta de clientes que desean rentar entretenimiento audiovisual. El sistema gestiona información de productos que incluyen películas individuales como unidad básica de renta con datos completos de nombre, director, duración en minutos, género cinematográfico, sinopsis descriptiva y precio de renta, formando los elementos fundamentales del inventario disponible para los clientes.

La arquitectura del sistema incorpora productos compuestos denominados sagas, que agrupan múltiples películas relacionadas temáticamente y pueden contener recursivamente otras sagas para formar estructuras jerárquicas complejas. Las sagas mantienen información consistente con películas individuales donde la duración total representa la suma acumulada de todos sus componentes, mientras que el precio aplicado recibe automáticamente un descuento del cinco por ciento sobre la suma total de precios individuales, incentivando la renta de colecciones completas sobre elementos aislados.

El sistema demuestra capacidad de composición anidada mediante ejemplos representativos como la Saga Trilogía Original de Star Wars que contiene tres películas individuales, mientras que estructuras más complejas como la Saga Colección Completa Star Wars incorpora múltiples sagas anidadas que a su vez contienen películas individuales, creando jerarquías profundas que el sistema maneja transparentemente mediante el patrón de diseño implementado.

Posteriormente, la expansión del negocio mediante la absorción de la cadena Mixdown especializada en venta de discos musicales introduce un sistema legacy que almacena información con estructura diferente incluyendo nombre del disco, artista intérprete, género musical, año de estreno y precio de venta. El requisito fundamental establece mantener el sistema legacy sin modificaciones estructurales, adaptándolo al catálogo existente mediante integración transparente que permite gestionar ambos tipos de productos en una única estructura unificada, aunque esta integración no permite diferenciar visualmente entre películas y discos musicales al nivel del catálogo.

El catálogo proporciona funcionalidades esenciales de consulta que incluyen visualización completa de todos los productos disponibles, filtrado selectivo por género cinematográfico o musical según preferencias del cliente, y filtrado económico mediante especificación de precio máximo para identificar opciones dentro del presupuesto disponible. La interfaz de consulta presenta inicialmente resumen simplificado mostrando únicamente nombre y precio de cada producto, permitiendo posteriormente selección interactiva de elementos específicos para desplegar información completa con todos los detalles almacenados en el sistema.

Análisis de Implementación de Patrones

2.1. Patrón Composite

¿Por qué se eligió esta implementación?

El patrón Composite fue implementado para gestionar la jerarquía compleja de productos del catálogo RockBuster, permitiendo tratar uniformemente elementos individuales como películas y estructuras compuestas como sagas mediante una interfaz común que facilita operaciones recursivas sobre composiciones anidadas arbitrariamente profundas. Esta elección se fundamenta en la necesidad fundamental de representar relaciones parte-todo donde las sagas contienen películas individuales y otras sagas, formando estructuras arbóreas que requieren navegación recursiva para operaciones como cálculo de

duración total, determinación de precio con descuento y reproducción secuencial de contenido.

La implementación se estructura mediante la clase abstracta `ProductoComponente` que define la interfaz común para todos los elementos del catálogo, estableciendo operaciones fundamentales mediante métodos abstractos `getNombre()`, `getGenero()`, `getPrecio()`, `getMinutosDuracion()`, `getSinopsis()` y `reproducir()` que garantizan tratamiento uniforme independientemente de la complejidad estructural del producto. Esta abstracción permite que código cliente manipule productos simples y compuestos mediante referencias polimórficas sin conocimiento de la estructura interna específica, maximizando flexibilidad y facilitando extensiones futuras del catálogo.

La clase `PeliculaIndividualHoja` implementa los elementos terminales del patrón Composite representando películas individuales como componentes atómicos que no contienen subcomponentes. Esta implementación almacena información específica incluyendo director, sinopsis descriptiva y duración en minutos, proporcionando implementaciones concretas de todos los métodos abstractos heredados que retornan valores directos sin necesidad de cálculos recursivos o agregación de datos desde componentes hijos.

Paralelamente, la clase `Saga` implementa el componente compuesto del patrón mediante gestión de una colección interna de elementos `ProductoComponente` que pueden ser películas individuales u otras sagas, demostrando la naturaleza recursiva fundamental del patrón. Esta implementación proporciona métodos de gestión estructural `agregarComponente()` y `removerComponente()` para manipular la composición dinámica, mientras que operaciones de consulta como `getMinutosDuracion()` y `getPrecioSaga()` implementan algoritmos recursivos que iteran sobre componentes hijos acumulando valores mediante programación funcional con streams.

El cálculo de duración total en sagas utiliza acumulación recursiva mediante iteración sobre todos los componentes invocando `getMinutosDuracion()` en cada elemento, aprovechando polimorfismo donde películas individuales retornan su duración específica mientras que sagas anidadas calculan recursivamente la suma de sus propios componentes. Este mecanismo proporciona transparencia completa donde la complejidad estructural permanece oculta al código cliente que simplemente invoca el método sin considerar la profundidad de anidación.

La determinación de precio con descuento implementa lógica especializada en sagas mediante `getPrecioSaga()` que suma precios de todos los componentes y aplica el porcentaje de descuento especificado, reflejando la política comercial que incentiva adquisición de colecciones completas. Esta funcionalidad demuestra cómo el patrón Composite facilita operaciones que requieren comportamiento diferenciado entre hojas y compuestos, permitiendo que sagas implementen lógica de negocio adicional mientras mantienen compatibilidad con la interfaz común.

El método `reproducir()` en sagas ilustra operaciones recursivas complejas mediante iteración sobre componentes invocando el mismo método en cada elemento, donde películas individuales retornan información de reproducción directa mientras que sagas anidadas reproducen recursivamente todos sus subcomponentes. Esta capacidad demuestra la potencia del patrón para manejar operaciones que deben aplicarse a toda la estructura jerárquica mediante delegación recursiva automática sin conocimiento explícito de la profundidad o complejidad estructural.

La integración del patrón en el sistema permite que la clase `Catalogo` gestione una única colección heterogénea de `ProductoComponente` que contiene simultáneamente películas individuales, sagas simples y sagas complejas con múltiples niveles de anidación. Esta unificación simplifica dramáticamente la gestión del catálogo al eliminar necesidad de colecciones separadas o lógica condicional para distinguir tipos, permitiendo que operaciones de filtrado, visualización y consulta procesen uniformemente toda la diversidad de productos mediante interfaces polimórficas consistentes.

2.2. Patrón Adapter

¿Por qué se eligió esta implementación?

El patrón Adapter fue implementado para integrar el sistema legacy de discos musicales de la cadena Mixdown dentro del catálogo unificado de RockBuster sin modificar la estructura original del código existente, proporcionando una capa de traducción que convierte la interfaz incompatible de `DiscoMusicalLegacy` hacia la interfaz esperada de `ProductoComponente` requerida por el resto del sistema. Esta elección se fundamenta en el requisito explícito de mantener el manejo original de información del sistema Mixdown mientras se logra interoperabilidad con la arquitectura establecida del catálogo RockBuster, evitando refactorización costosa del código legacy y preservando funcionalidad existente probada en producción.

La implementación se materializa mediante la clase `DiscoMusicalAdapter` que extiende `ProductoComponente` estableciendo compatibilidad estructural con el resto del catálogo, mientras mantiene internamente una instancia de `DiscoMusicalLegacy` que encapsula la representación original de discos musicales con sus atributos específicos de artista, género musical, año de estreno y precio de venta. Esta relación de composición permite que el adaptador delegue operaciones al objeto legacy mientras traduce llamadas entre las interfaces incompatibles, actuando como intermediario transparente que oculta diferencias estructurales al código cliente.

La clase legacy `DiscoMusicalLegacy` mantiene su implementación original con métodos específicos como `getArtista()`, `getGeneroMusical()`, `getAñoEstreno()` y `getPrecioVenta()` que reflejan el dominio musical distinto del dominio cinematográfico de películas. El sistema preserva esta estructura sin modificaciones permitiendo que código existente de Mixdown continúe funcionando sin cambios, mientras que el adaptador proporciona la traducción necesaria para integración con el catálogo unificado que espera interfaz de `ProductoComponente`.

La traducción de interfaz se implementa mediante sobrescritura de métodos abstractos heredados de `ProductoComponente` donde el adaptador proporciona implementaciones que delegan al objeto legacy cuando existe correspondencia directa, o retornan valores apropiados para métodos sin equivalente en el sistema musical. El método `reproducir()` delega directamente a `discoLegacy.reproducir()` aprovechando funcionalidad existente, mientras que `getNombre()`, `getGenero()` y `getPrecio()` se inicializan durante construcción del adaptador mapeando desde los valores correspondientes del disco legacy.

Los métodos sin correspondencia natural en el dominio musical como `getMinutosDuracion()` y `getSinopsis()` implementan comportamiento por defecto retornando cero para duración y null para sinopsis, reconociendo que estos conceptos cinematográficos no aplican directamente a discos musicales. Esta decisión de diseño mantiene compatibilidad de interfaz permitiendo que discos musicales participen en operaciones generales del catálogo, mientras que código que requiere específicamente duración o sinopsis puede verificar valores por defecto e implementar comportamiento alternativo apropiado.

El método `obtenerInformacion()` proporciona acceso a la funcionalidad completa del sistema legacy mediante delegación a `discoLegacy.obtenerDetallesCompleto()`, permitiendo que código que conoce específicamente el tipo adaptador acceda a información musical detallada incluyendo artista, género musical y año de estreno que no tiene representación directa en la interfaz general de `ProductoComponente`. Esta capacidad demuestra cómo el adaptador puede exponer funcionalidad adicional del sistema legacy cuando sea apropiado, sin comprometer la compatibilidad fundamental con la interfaz común.

El método `toString()` implementa comportamiento especializado delegando a `discoLegacy.obtenerDetallesCompleto()` para proporcionar representación textual que incluye todos los atributos musicales específicos, preservando la riqueza informativa del sistema original. Esta implementación permite que operaciones de depuración, logging y visualización accedan a información completa del disco musical sin pérdida de detalles durante la adaptación.

La integración del patrón en el sistema permite que la clase `Catalogo` almacene uniformemente películas individuales, sagas compuestas y discos musicales adaptados en una única colección de `ProductoComponente`, eliminando necesidad de lógica condicional para distinguir tipos o colecciones separadas para diferentes categorías de productos. Las operaciones de filtrado por género y precio máximo funcionan transparentemente sobre discos musicales adaptados gracias a la interfaz común, mientras que el sistema de visualización puede acceder tanto a información general mediante métodos heredados como a detalles específicos mediante casting cuando sea necesario.

La arquitectura resultante demuestra composición efectiva de patrones donde Composite proporciona la estructura jerárquica flexible para productos cinematográficos mientras que Adapter permite integración sin fricción de productos musicales del sistema legacy, creando un catálogo unificado heterogéneo que gestiona transparentemente múltiples tipos de entretenimiento mediante interfaces polimórficas consistentes que facilitan extensibilidad futura y mantenimiento simplificado.

Estado de la Implementación

3.1. Componentes Implementados

La práctica se encuentra **completamente implementada** con todos los componentes requeridos y funcionalidades especificadas en el enunciado original. El desarrollo incluye la correcta implementación de los dos patrones de diseño requeridos (Composite y Adapter), la integración completa del sistema de catálogo RockBuster con capacidades de gestión de productos heterogéneos, y un sistema de consulta interactivo que permite filtrado por género y precio máximo según las especificaciones establecidas.

El sistema incorpora estructuras de datos que reflejan fielmente los requerimientos del enunciado, utilizando una única lista unificada que almacena doce productos variados demostrando la diversidad requerida del catálogo. La colección incluye cuatro películas individuales de Star Wars disponibles para renta independiente con información completa de director, sinopsis, duración y precio, cuatro sagas compuestas que demuestran capacidad de anidación múltiple incluyendo la Trilogía Original, Trilogía de Precuelas, Trilogía de Secuelas y una Saga Colección Completa que anida recursivamente las tres trilogías anteriores, y cuatro discos musicales adaptados del sistema legacy Mixdown representando géneros diversos de Rock, Pop, Jazz y Música Clásica.

Las películas individuales implementan correctamente todos los atributos especificados almacenando nombre descriptivo, director responsable de la producción, duración en minutos, género cinematográfico, sinopsis narrativa completa y precio de renta en unidades monetarias. Los ejemplos incluyen producciones icónicas como Star Wars Episodio IV Una Nueva Esperanza dirigida por George Lucas con 121 minutos de duración y precio de 45.00, Star Wars Episodio V El Imperio Contraataca dirigida por Irvin Kershner con 124 minutos y precio de 47.00, y Rogue One Una Historia de Star Wars dirigida por Gareth Edwards con 133 minutos y precio de 55.00.

Las sagas compuestas implementan correctamente el patrón Composite mediante gestión de colecciones internas de componentes que pueden ser películas individuales u otras sagas anidadas. La Saga Trilogía Original contiene tres películas individuales con duración total calculada recursivamente de 376 minutos y precio con descuento de 131.10 aplicando el cinco por ciento de reducción sobre la suma de componentes. La Saga Colección Completa Star Wars demuestra anidación profunda conteniendo tres sagas que a su vez contienen múltiples películas, calculando automáticamente duración total acumulada y precio con descuento aplicado recursivamente a toda la jerarquía.

Los discos musicales adaptados integran exitosamente el sistema legacy mediante el patrón Adapter, preservando información original de artista, género musical y año de estreno mientras proporcionan compatibilidad con la interfaz común del catálogo. Los ejemplos incluyen The Dark Side of the Moon de Pink Floyd del género Rock Progresivo estrenado en 1973 con precio de 35.00, Thriller de Michael Jackson del género Pop estrenado en 1982 con precio de 32.00, Kind of Blue de Miles Davis del género Jazz estrenado en 1959 con precio de 28.00, y Symphony No. 9 de Ludwig van Beethoven del género Clásica estrenado en 1824 con precio de 25.00, demostrando diversidad musical significativa.

3.2. Estructura del Proyecto

El proyecto está organizado arquitectónicamente mediante una estructura de paquetes que refleja la separación clara de responsabilidades según los principios de diseño orientado a objetos y facilita el mantenimiento del código. El paquete raíz contiene la clase principal Main que actúa como punto de entrada del sistema, coordinando la inicialización del catálogo con productos predefinidos y delegando la ejecución a componentes especializados mediante programación funcional pura que maximiza composición de funciones.

La implementación del patrón Composite se estructura dentro del paquete Composite que contiene la clase abstracta `ProductoComponente` definiendo la interfaz común para todos los productos del catálogo, la clase `PeliculaIndividualHoja` implementando elementos terminales que representan películas individuales sin subcomponentes, y la clase `Saga` implementando elementos compuestos que gestionan colecciones de productos mediante métodos de agregación, remoción y consulta recursiva que calculan duración total y precio con descuento aplicado.

El patrón Adapter se organiza dentro del paquete Adapter que encapsula la integración del sistema legacy mediante la clase `DiscoMusicalLegacy` preservando la estructura original del sistema Mixdown con atributos específicos de artista, género musical, año de estreno y precio de venta, junto con la clase `DiscoMusicalAdapter` que extiende `ProductoComponente` proporcionando traducción de interfaz mediante delegación al objeto legacy encapsulado mientras implementa métodos abstractos heredados con comportamiento apropiado para productos musicales.

El modelo de dominio se estructura mediante el paquete Modelo que contiene la clase `Catalogo` responsable de gestionar la colección unificada de productos heterogéneos proporcionando operaciones de consulta como visualización completa, filtrado por género y filtrado por precio máximo mediante inyección de dependencias de filtros especializados. El paquete también incluye la clase `MenuInteractivo` que coordina la interfaz de usuario mediante presentación de opciones, captura de entrada del usuario y delegación a operaciones apropiadas del catálogo según la selección realizada.

El sistema de filtrado implementa el patrón Strategy mediante el paquete Filtros que contiene la interfaz `FiltroProducto` definiendo el contrato común para estrategias de filtrado, la clase `FiltroGenero` implementando filtrado selectivo por género mediante comparación de strings ignorando mayúsculas, y la clase `FiltroPrecioMaximo` implementando filtrado económico mediante comparación numérica que retiene productos con precio menor o igual al máximo especificado, permitiendo intercambio dinámico de estrategias sin modificar código cliente.

La visualización de productos se organiza mediante el paquete Visualizar que contiene la interfaz `VisualizadorProducto` definiendo operaciones de presentación de información, y la clase `VisualizadorConsola` implementando visualización específica para terminal de texto mediante formateo apropiado que presenta resumen simplificado con nombre y precio para listados generales, y despliegue de información completa incluyendo todos los atributos específicos cuando se selecciona un producto individual para consulta detallada.

La arquitectura modular resultante facilita extensibilidad futura donde nuevos tipos de productos pueden integrarse mediante implementación de `ProductoComponente` o creación de nuevos adaptadores para sistemas legacy adicionales, mientras que nuevas estrategias de filtrado y visualización pueden agregarse mediante implementación de interfaces correspondientes sin impactar componentes existentes, demostrando adherencia a principios SOLID que maximizan mantenibilidad y minimizan acoplamiento entre módulos.

Funcionamiento de la Práctica

El sistema opera mediante un flujo operativo integral que coordina meticulosamente la gestión del catálogo de productos RockBuster, proporcionando una experiencia interactiva completa para consulta de películas, sagas y discos musicales mediante implementación armoniosa de los patrones Composite y Adapter que facilitan operaciones uniformes sobre productos heterogéneos con estructuras jerárquicas complejas. La operación inicia con la ejecución de la clase `Main` que invoca métodos de inicialización implementados mediante programación funcional pura, creando todos los productos predefinidos del catálogo y estableciendo las dependencias necesarias para el funcionamiento del sistema interactivo.

La fase de inicialización ejecuta `crearProductosIniciales()` que coordina la construcción de doce productos mediante invocación de métodos especializados `crearPeliculasIndividuales()`, `crearSagasAnidadas()` y `crearDiscosMusicalesAdaptados()`, cada uno responsable de instanciar una categoría específica de productos con datos completos y realistas. El proceso utiliza streams funcionales para combinar las colecciones resultantes en una única lista unificada que se pasa al constructor de `Catalogo` junto con una

instancia de `VisualizadorConsola`, estableciendo la configuración inicial completa del sistema mediante composición pura de funciones.

La creación de películas individuales instancia cuatro objetos `PeliculaIndividualHoja` con información detallada de producciones Star Wars incluyendo nombres descriptivos, directores reconocidos, duraciones específicas en minutos, género de Ciencia Ficción, sinopsis narrativas que resumen la trama, y precios de renta diferenciados reflejando variabilidad comercial. Estos elementos forman las hojas del patrón `Composite` que posteriormente se integran en sagas compuestas o permanecen disponibles como productos independientes para renta individual según preferencias del cliente.

La construcción de sagas anidadas demuestra la capacidad recursiva del patrón `Composite` mediante creación de tres sagas simples que contienen películas individuales (Trilogía Original, Trilogía de Precuelas, Trilogía de Secuelas), seguida de la creación de una saga compleja (Colección Completa Star Wars) que contiene las tres sagas anteriores como componentes, formando una estructura jerárquica de tres niveles donde la saga completa contiene sagas que contienen películas. El proceso utiliza programación funcional con `forEach` para agregar componentes a cada saga, aprovechando el método `agregarComponente()` que gestiona la colección interna de forma inmutable.

La integración de discos musicales adaptados instancia cuatro objetos `DiscoMusicalAdapter` que encapsulan instancias correspondientes de `DiscoMusicalLegacy` con información musical completa de artista, género musical específico, año de estreno histórico y precio de venta. Esta construcción demuestra el patrón `Adapter` donde cada adaptador recibe un objeto `legacy` en su constructor y proporciona compatibilidad con la interfaz `ProductoComponente`, permitiendo que discos musicales participen transparentemente en todas las operaciones del catálogo sin modificar el código original del sistema `Mixdown`.

Una vez completada la inicialización, el sistema transfiere control a `MenuInteractivo` que presenta un bucle principal de interacción mediante despliegue de opciones numeradas que incluyen visualizar catálogo completo, filtrar productos por género, filtrar productos por precio máximo, mostrar información completa de un producto específico y salir de la aplicación. El menú captura entrada del usuario mediante `Scanner` y ejecuta la operación correspondiente mediante delegación al objeto `Catalogo`, implementando separación clara entre presentación de interfaz y lógica de negocio.

La operación de visualización completa invoca `verCatalogoCompleto()` que retorna la lista completa de productos sin filtrado, delegando posteriormente al `VisualizadorConsola` para presentar cada producto mediante formato simplificado que muestra únicamente nombre y precio en líneas numeradas secuencialmente. Esta presentación permite que usuarios evalúen rápidamente opciones disponibles sin sobrecarga de información detallada, facilitando navegación eficiente del catálogo mediante escaneo visual rápido de nombres y precios.

El filtrado por género solicita al usuario especificar el género deseado mediante entrada de texto, invocando `filtrarPorGenero()` en el catálogo que delega a la estrategia `FiltroGenero` para evaluar cada producto comparando su género con el criterio especificado ignorando diferencias de mayúsculas. El proceso utiliza streams funcionales con operación `filter` que retiene únicamente productos coincidentes, retornando una lista reducida que se presenta mediante el visualizador con formato idéntico al catálogo completo, permitiendo que usuarios localicen productos de su género preferido sin navegar manualmente toda la colección.

El filtrado por precio máximo solicita al usuario especificar el límite presupuestario mediante entrada numérica, invocando `filtrarPorPrecioMaximo()` que delega a la estrategia `FiltroPrecioMaximo` para evaluar cada producto comparando su precio con el máximo especificado. La implementación utiliza streams funcionales con `filter` que retiene productos con precio menor o igual al límite, retornando una lista económicamente viable que se presenta mediante visualización estándar, permitiendo que usuarios con restricciones presupuestarias identifiquen opciones accesibles sin considerar manualmente cada precio individual.

La visualización de información completa solicita al usuario seleccionar un producto específico mediante entrada del número correspondiente en el listado previamente presentado, recuperando el ob-

jeto `ProductoComponente` de la lista actual e invocando `mostrarProductoCompleto()` que delega al `VisualizadorConsola` para presentar todos los atributos disponibles. Para películas individuales el sistema despliega director, duración, género, sinopsis completa y precio, para sagas presenta lista de componentes con duración total calculada recursivamente y precio con descuento aplicado, y para discos musicales adaptados muestra artista, género musical, año de estreno y precio, demostrando cómo el polimorfismo permite presentación especializada según el tipo concreto del producto.

La operación de reproducción disponible desde el menú de información completa invoca el método `reproducir()` del producto seleccionado, donde películas individuales retornan mensaje formateado con información de reproducción, sagas ejecutan reproducción recursiva de todos sus componentes mediante iteración sobre la colección interna invocando `reproducir` en cada elemento, y discos musicales adaptados delegan al objeto `legacy` que retorna mensaje con información del artista y año de estreno. Esta funcionalidad demuestra cómo el patrón `Composite` facilita operaciones recursivas complejas que se aplican transparentemente a toda la jerarquía estructural mediante delegación automática.

El flujo completo de operación del sistema permite experimentar interactivamente con todos los aspectos implementados, desde la gestión unificada de productos heterogéneos hasta la aplicación de filtros selectivos y visualización especializada según tipos, proporcionando una demostración integral de la funcionalidad especificada para el catálogo `RockBuster` completamente operativo. El sistema culmina retornando al menú principal después de cada operación permitiendo ejecución secuencial de múltiples consultas, o finalizando limpiamente mediante la opción de salida que termina el bucle de interacción y permite terminación ordenada del programa con liberación apropiada de recursos.

Requisitos del Sistema

5.1. Versión de Java

La implementación de esta práctica requiere específicamente **Java Development Kit (JDK) versión 17** para su correcta compilación y ejecución. Esta selección tecnológica se fundamenta en las características que proporciona Java 17 como versión de soporte a largo plazo (LTS), garantizando un entorno de desarrollo robusto y confiable para la implementación de patrones de diseño complejos y la gestión de sistemas orientados a objetos sofisticados que requieren el manejo coordinado de estructuras jerárquicas mediante composición recursiva y adaptación de sistemas `legacy` heterogéneos.

Java 17 introduce optimizaciones significativas en el rendimiento de la máquina virtual que benefician directamente la ejecución de los patrones implementados en el sistema de catálogo `RockBuster`. Las mejoras en la gestión de memoria y el `garbage collection` optimizado facilitan el manejo eficiente de las múltiples instancias de objetos que se crean durante la construcción del catálogo, especialmente durante la instanciación de sagas anidadas que generan estructuras jerárquicas complejas con múltiples niveles de composición donde cada componente requiere asignación de memoria independiente.

Las optimizaciones en el manejo de polimorfismo y `dispatch` de métodos virtuales mejoran significativamente el rendimiento durante operaciones recursivas en sagas compuestas, donde cada invocación a métodos como `getMinutosDuracion()` o `reproducir()` debe determinar dinámicamente el tipo concreto del componente y delegar apropiadamente. Estas mejoras son particularmente relevantes durante operaciones que navegan jerarquías profundas de sagas anidadas, donde el sistema ejecuta múltiples resoluciones polimórficas en cascada para calcular agregaciones recursivas de duración y precio.

La versión seleccionada también proporciona mejor soporte para estructuras de datos inmutables y programación funcional mediante `streams`, aspectos fundamentales utilizados durante la inicialización del catálogo donde métodos como `crearProductosIniciales()` emplean operaciones funcionales de `flatMap` y `collect` para combinar colecciones heterogéneas en una única lista unificada. Java 17 incluye optimizaciones específicas para estas operaciones que mejoran el rendimiento durante la fase de inicialización del sistema.

Adicionalmente, Java 17 incluye mejoras en el sistema de inferencia de tipos y manejo de genéricos que facilitan la implementación del patrón `Composite` con colecciones fuertemente tipadas, donde la

lista de componentes en Saga utiliza `List<ProductoComponente>` garantizando type safety en tiempo de compilación. Estas características contribuyen a la robustez del código reduciendo posibilidad de errores de tipos y facilitando refactorización segura cuando sea necesario extender funcionalidad del catálogo.

Compilación y Ejecución del Proyecto

6.1. Proceso de Compilación

La compilación del proyecto se realiza mediante el compilador estándar de Java `javac`, aprovechando la estructura modular de paquetes que organiza las clases según sus responsabilidades funcionales específicas dentro del sistema de catálogo RockBuster. El proceso requiere la navegación al directorio `src` del proyecto, desde donde se ejecutan las instrucciones de compilación que procesan recursivamente toda la jerarquía de clases del sistema ubicadas en el subdirectorio `main` que contiene todos los paquetes implementados.

La arquitectura del proyecto demanda una compilación ordenada que respete las dependencias entre las diferentes clases y paquetes, comenzando por las clases abstractas e interfaces fundamentales como `ProductoComponente`, `FiltroProducto` y `VisualizadorProducto`, progresando hacia las implementaciones concretas que materializan los patrones de diseño implementados. El sistema de compilación debe procesar las abstracciones antes de compilar sus implementaciones concretas, asegurando que todas las referencias se resuelvan correctamente durante el proceso de generación de bytecode optimizado.

Para efectuar la compilación completa del sistema, se debe posicionar en el directorio `src` del proyecto y ejecutar la instrucción de compilación que utiliza comodines de shell para procesar recursivamente todos los archivos Java distribuidos en la estructura de paquetes bajo el directorio `main`. El comando específico para compilar todos los archivos del sistema es:

```
javac main/**/*.java main/*.java
```

Este proceso se realiza mediante la aplicación sistemática del compilador `javac` utilizando el patrón `glob main/**/*.java` que instruye al compilador para procesar recursivamente todos los archivos con extensión `.java` en cualquier subdirectorio bajo `main`, respetando la jerarquía de paquetes establecida durante la fase de desarrollo. El patrón adicional `main/*.java` asegura compilación de archivos ubicados directamente en el directorio `main` sin estar en subdirectorios específicos de paquetes.

La compilación exitosa genera archivos de bytecode con extensión `.class` que mantienen la misma estructura jerárquica de directorios que el código fuente, permitiendo que la máquina virtual de Java localice y cargue correctamente todas las clases durante la ejecución posterior utilizando el mecanismo de `classpath`. Este proceso produce archivos optimizados que aprovechan las características específicas de Java 17 para proporcionar rendimiento óptimo durante la ejecución del sistema de catálogo con operaciones recursivas sobre jerarquías complejas de productos.

Es importante señalar que la compilación se ejecuta desde el directorio `src` del proyecto en sistemas Linux, donde el shell expande apropiadamente los patrones `glob` especificados en el comando. Esta ubicación específica es crítica para que el compilador resuelva correctamente las referencias entre paquetes y genere la estructura de directorios apropiada para el bytecode resultante que posteriormente será cargado por la máquina virtual durante la fase de ejecución.

6.2. Ejecución del Sistema

La ejecución del sistema se inicia mediante el comando estándar de Java `java`, especificando la clase principal `Main` que actúa como punto de entrada de toda la aplicación y coordina la inicialización completa del sistema de catálogo RockBuster con todos sus productos predefinidos. La ejecución debe realizarse desde el directorio `src/main` del proyecto, donde se encuentran disponibles los archivos bytecode compilados organizados según la estructura de paquetes establecida durante la fase de compilación.

El comando de ejecución específico para iniciar el sistema completo utiliza el siguiente formato, ejecutado desde el directorio `src/main`:

```
java Main
```

Este comando simple invoca la máquina virtual de Java que localiza y carga la clase `Main` desde el directorio actual, ejecutando su método `main(String[] args)` que constituye el punto de entrada estándar de aplicaciones Java. La máquina virtual resuelve automáticamente todas las dependencias del sistema cargando las clases necesarias desde los paquetes `Composite`, `Adapter`, `Modelo`, `Visualizar` y `Filtros` según sean requeridas durante la ejecución del flujo operativo del programa.

Durante la ejecución, el sistema presenta información interactiva en la consola mostrando el menú principal con opciones numeradas que permiten al usuario consultar el catálogo completo, aplicar filtros por género o precio máximo, visualizar información detallada de productos específicos y reproducir contenido seleccionado. La interfaz captura entrada del usuario mediante `Scanner` permitiendo navegación interactiva por todas las funcionalidades implementadas, demostrando operaciones sobre productos heterogéneos mediante los patrones `Composite` y `Adapter`.

El flujo de ejecución demuestra la aplicación del patrón `Composite` mediante operaciones recursivas sobre sagas anidadas, donde consultas de duración total o precio con descuento navegan automáticamente toda la jerarquía de componentes sin requerir lógica condicional explícita en el código cliente. Similarmente, la integración de discos musicales mediante el patrón `Adapter` permite que productos del sistema legacy participen transparentemente en todas las operaciones del catálogo, demostrando traducción exitosa de interfaces incompatibles mediante la capa de adaptación implementada.

La salida en tiempo real permite supervisar el funcionamiento correcto de todos los componentes implementados, evidenciando la gestión unificada de películas individuales, sagas compuestas y discos musicales adaptados mediante una única interfaz polimórfica. El sistema permite experimentar con filtrado selectivo que aplica criterios uniformemente sobre todos los tipos de productos, y visualización especializada que presenta información apropiada según el tipo concreto del producto seleccionado mediante resolución dinámica de métodos polimórficos.

El flujo completo de operación del sistema permite experimentar interactivamente con todos los aspectos implementados, desde la construcción de jerarquías complejas mediante composición recursiva hasta la integración transparente de sistemas legacy mediante adaptación de interfaces, proporcionando una demostración integral de la funcionalidad especificada para el catálogo `RockBuster` completamente operativo. El sistema culmina presentando un menú persistente que permite múltiples operaciones secuenciales, finalizando limpiamente mediante la opción de salida que termina el bucle de interacción y permite terminación ordenada del programa con liberación apropiada de recursos del sistema.